

CHAPTER 25

Clustering

Clustering is an unsupervised machine learning technique for grouping homogeneous data points into partitions called clusters. In the example dataset illustrated in Figure 25-1, suppose we have a set of n points and 2 features. A clustering algorithm can be applied to determine the number of distinct subclasses or groups among the data samples.

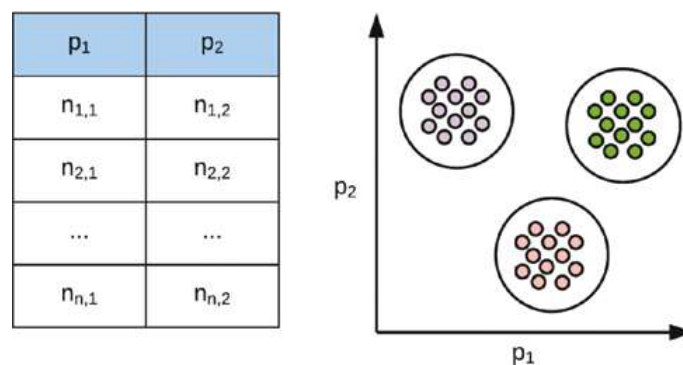


Figure 25-1. An illustration of clustering in a 2-D space

Clustering a 2-D dataset as seen in Figure 25-1 is relatively trivial. The real challenge arises when we have to perform clustering in higher-dimensional spaces. The question now is how do we ascertain or find out if a set of points are similar or if a set of points should be in the same group? In this section, we would cover two essential types of clustering algorithms known as k-means clustering and hierarchical clustering.

K-means clustering is used when the number of anticipated distinct classes or sub-groups is known in advance. In hierarchical clustering, the exact number of clusters is not known, and the algorithm is tasked to find the optimal number of heterogeneous sub-groups in the dataset.

K-Means Clustering

k-Means clustering is one of the most famous and widely used clustering algorithms in practice. It works by using a distance measurement (most commonly the Euclidean distance) to iteratively assign data points in a hyperspace to a set of non-overlapping clusters.

In K -means, the anticipated number of clusters, K , is chosen at the onset. The clusters are initialized by arbitrarily selecting at random one of the data points as an initial cluster for each K . The algorithm now works by iteratively assigning each point in the space to the cluster centroid that it is nearest to using the distance measurement.

After all the points have been assigned to their closest cluster point, the cluster centroid is adjusted to find a new center among the points in the cluster. This process is repeated until the algorithm converges, that is, when the cluster centroids stabilize and points do not readily swap clusters after every reassignment. These steps are illustrated in Figure 25-2.

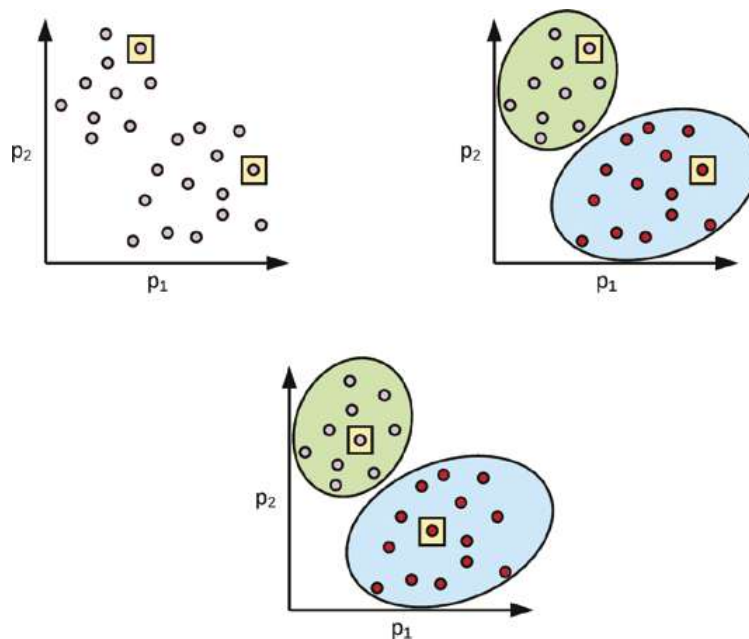


Figure 25-2. An illustration of k -means clustering with $k = 2$. Top left: Randomly pick a point for each k . Top right: Iteratively assign each point to its closest cluster centroid. Bottom: Update the cluster centroids for each of the k clusters. Typically, we repeat the iterative assignment of all the points and update the cluster centroid until the algorithm resolves in a stable clustering.

Considerations for Selecting K

There's really no way of telling the number of clusters in a dataset from the onset. The best way of selecting k is to try out different values of K to see what works best in creating distinct clusters.

Another strategy, which is widely employed in practice, is to compute the average distance of the points in the cluster to the cluster centroid for all clusters. This estimate is plotted on a graph as we progressively increase the value of K . We observe that as K increases, the distance of points from the centroid of its cluster gradually reduces, and the generated curve resembles the elbow of an arm. From practice, we choose the value of K just after the elbow as the best K value for that dataset. This method is called the elbow method for selecting K as is illustrated in Figure 25-3.

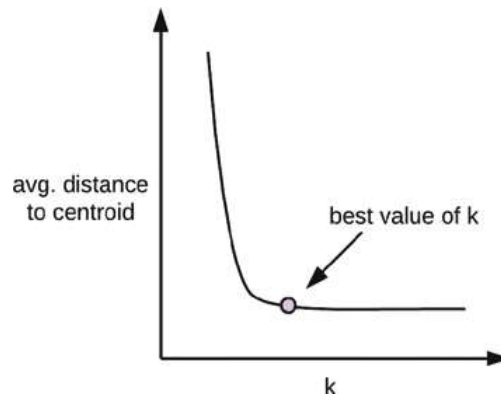


Figure 25-3. The elbow method for choosing the best value of k

Considerations for Assigning the Initial K Points

The points that determine the initial value of K are important in finding a good set of clusters. By selecting the point for K at random, two or more points may reside in the same cluster, and this will invariably lead to sub-par results. To mitigate this from occurring, we can employ more sophisticated approaches to selecting the value of K . A common strategy is to randomly select the first K point and then select the next point as the point that is farthest from the first chosen point. This strategy is repeated until all K points have been selected. Another approach is to run hierarchical clustering on a sub-sample of the dataset (this is because hierarchical clustering is a computationally expensive algorithm) and use the number of clusters after cutting off the dendrogram as the value of K .

K-Means Clustering with Scikit-learn

This example implements K-means clustering with Scikit-learn. Since this is an unsupervised learning use case, we use just the features of the Iris dataset to cluster the observations into labels.

```
# import packages
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn import datasets
from sklearn.model_selection import train_test_split

# load dataset
data = datasets.load_iris()

# get the dataset features
X = data.data

# create the model. Since we know that the Iris dataset has 3 classes, we
set n_clusters = 3
kmeans = KMeans(n_clusters=3, random_state=0)

# fit the model on the training set
kmeans.fit(X)

'Output':
KMeans(algorithm='auto', copy_x=True, init='k-means++', max_iter=300,
       n_clusters=3, n_init=10, n_jobs=1, precompute_distances='auto',
       random_state=0, tol=0.0001, verbose=0)

# predict the closest cluster each sample in X belongs to.
y_kmeans = kmeans.predict(X)

# plot clustered labels
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, cmap='viridis')

# plot cluster centers
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='black', s=200, alpha=0.7);
plt.show()
```

The code to plot the clustered labels and the cluster centers should be executed in the same notebook. The plot of clusters made by the K-means algorithm is shown in Figure 25-4.

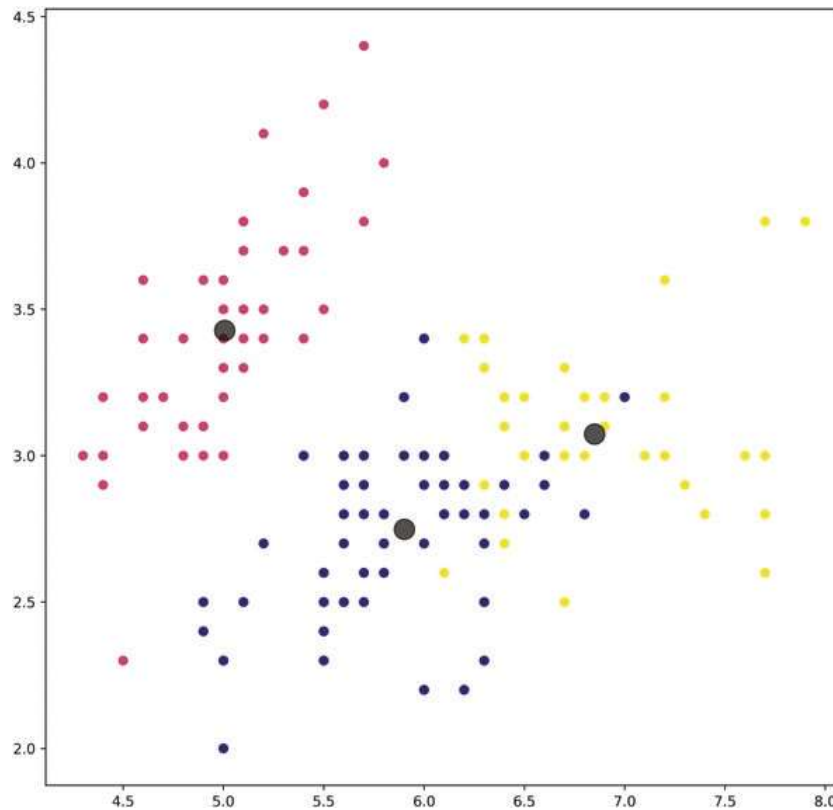


Figure 25-4. Plot of K-means clusters and their cluster centers

Hierarchical Clustering

Hierarchical clustering is another clustering algorithm for finding homogeneous sub-groups or classes within a dataset. However, as opposed to k -means, we do not need to make an a priori assumption of the number of clusters in the dataset before running the algorithm.

The two main techniques for performing hierarchical clustering are

- Bottom-up or agglomerative
- Top-down or divisive

In the bottom-up or agglomerative method, each data point is initially designated as a cluster. Clusters are iteratively combined based on homogeneity that is determined by some distance measure. On the other hand, the divisive or top-down approach starts with a cluster and subsequently splits into homogeneous sub-groups.

Hierarchical clustering creates a tree-like representation of the partitioning called a dendrogram. A dendrogram is drawn somewhat similar to a binary tree with the root at the top and the leaves at the bottom. The leaf on the dendrogram represents a data sample. The dendrogram is constructed by iteratively combining the leaves based on homogeneity to form clusters moving up the tree. An illustration of hierarchical clustering is shown in Figure 25-5.

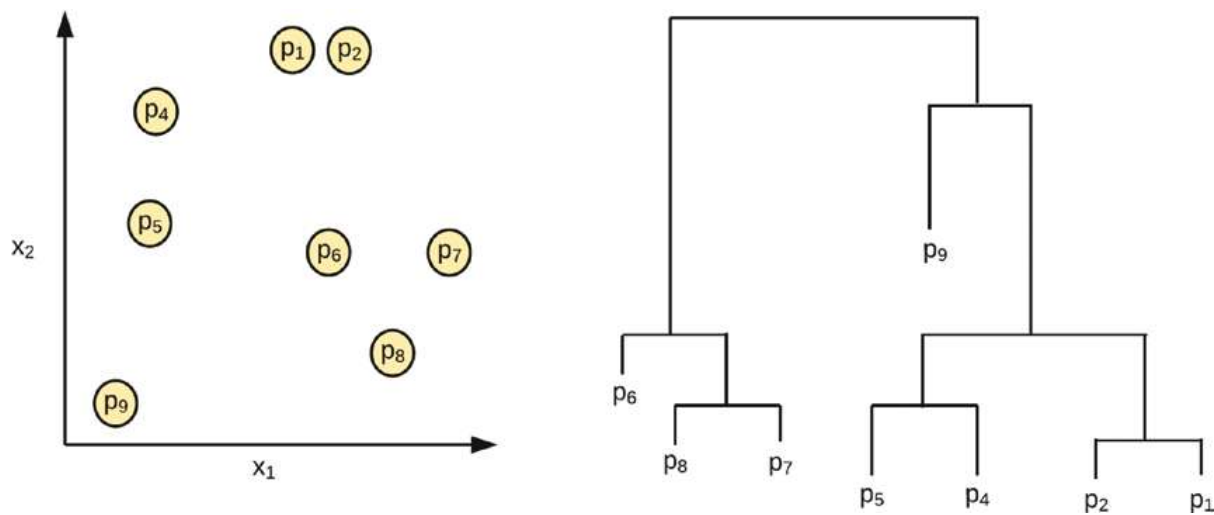


Figure 25-5. An illustration of hierarchical clustering of data points in a 2-D feature space. Left: The spatial representation of points in 2-D space. Right: A hierarchical cluster of points represented by a dendrogram.

How Are Clusters Formed

Clusters are formed by computing the nearness between each pair of data points. The notion of nearness is most popularly calculated using the Euclidean distance measure. Beginning at the leaves of the dendrogram, we iteratively combine those data points that are closer to one another in the multi-dimensional vector space until all the homogeneous points are placed into a single group or cluster.

The Euclidean distance is used to compute the nearness between n data points. After each pair of data points has combined to form a cluster, the new cluster pairs are then pulled into groups going up the tree, with the tree branch or dendrogram height reflecting the dissimilarity between the clusters.

Dissimilarity computes how different each cluster of data is from one another. The notion of dissimilarity between two clusters or groups is described in terms of *linkage*. Four types of linkage exist for grouping clusters in hierarchical clustering. They are centroid, complete, average, and single.

The centroid linkage computes the dissimilarity between two clusters using the geometric centroid of the clusters. The complete linkage uses the two farthest data points between the two clusters to compute the dissimilarity (see Figure 25-6).

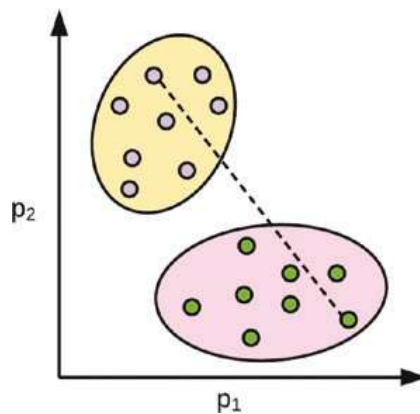


Figure 25-6. Complete linkage

The average linkage finds the means of points within the pair of clusters and uses that new artificial point to calculate the dissimilarity (see Figure 25-7).

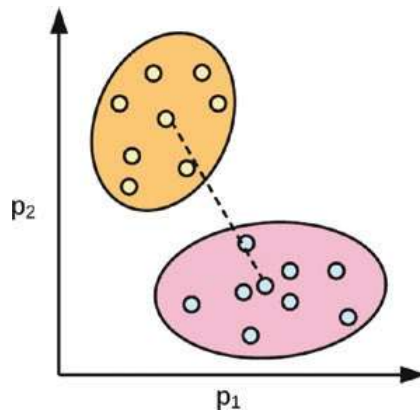


Figure 25-7. Average linkage

The single linkage uses the closest data point between the cluster pairs to compute the dissimilarity measure (see Figure 25-8).

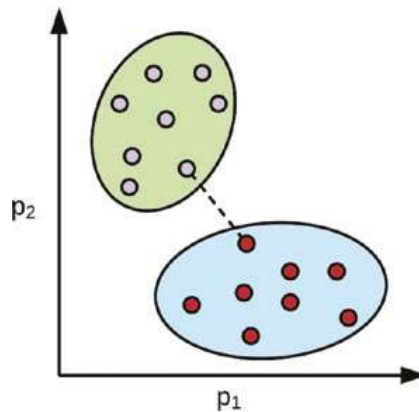


Figure 25-8. *Single linkage*

Empirically, the complete and average linkages are preferred in practice because they yield more balanced dendrograms. Other dissimilarity measures exist for evaluating the nearness or homogeneity of data points. One of such is the Manhattan distance, another distance-based measure, or the correlation-based distance which groups pairs of data samples with highly correlated features. A correlated-based dissimilarity measure may be more useful in datasets where proximity in multi-dimensional spaces is not as useful a metric for homogeneity as compared to the correlation of their features in the space. A choice of calculating dissimilarity has a significant impact on the ensuring dendrogram.

After running the algorithm, the dendrogram is cut at a particular height, and the number of distinct lines or branches after the cut is circumscribed as the number of clusters in the dataset. An illustration of cutting the dendrogram is shown in Figure 25-9.

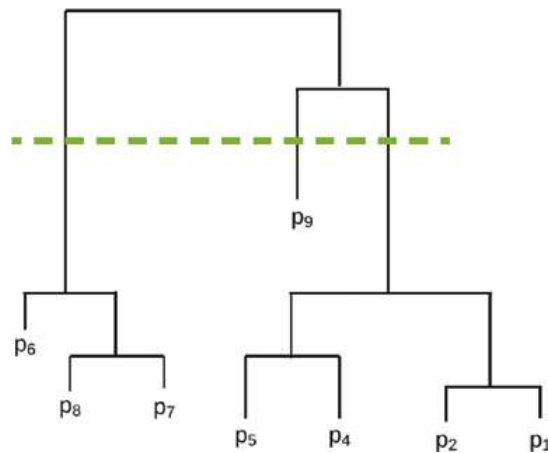


Figure 25-9. *Dendrogram cut*

Hierarchical Clustering with the SciPy Package

This example implements hierarchical or agglomerative clustering with SciPy. The ‘`scipy.cluster.hierarchy`’ package has simple methods for performing hierarchical clustering and plotting dendrograms. This example uses the ‘complete’ linkage method. The plot of the dendrogram is shown in [Figure 25-10](#).

```
# import packages
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram
from scipy.cluster import hierarchy

Z = hierarchy.linkage(X, method='complete')

plt.figure()
dn = hierarchy.dendrogram(Z, truncate_mode='lastp')
```

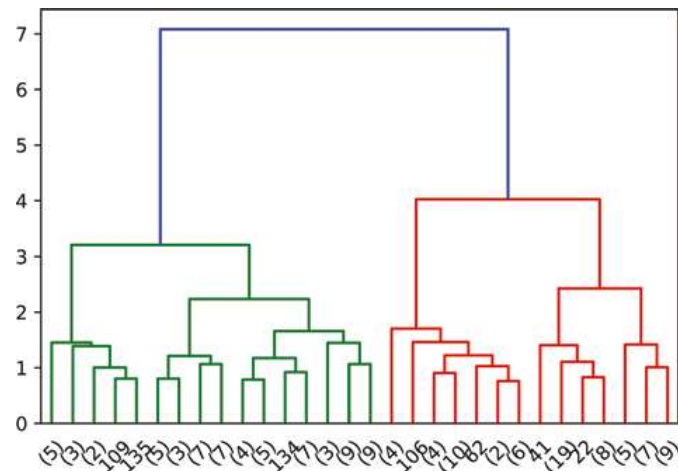


Figure 25-10. Dendrogram produced by hierarchical clustering

This chapter reviewed the pros and cons of K -means and hierarchical clustering. Both hierarchical and K -means are susceptible to perturbations in the dataset and can give very different results if a few data points are removed or added. Also, it is crucial to standardize the dataset features (i.e., to subtract each element in the feature from its mean and divide by its standard deviation or by the range) before performing clustering. This ensures that the features are within similar numeric bounds and have tempered or measured distances in the feature space.

The results of these clustering algorithms also depend on a wide range of considerations such as the choice of K for K -means, and for hierarchical clustering, the choice of dissimilarity measure, the type of linkage, and where to cut the dendrogram all affect the final result of the clusters. Hence, to get the best out of clustering, it is best to perform a grid search and try out all these different configurations in order to get a measured view on the robustness of the results before applying into your learning pipeline or using as a model to explain the dataset.

In the next chapter, we will discuss principal component analysis (PCA) as an unsupervised machine learning algorithm for finding low-dimensional feature subspaces that capture the variability in the dataset.

CHAPTER 26

Principal Component Analysis (PCA)

Principal component analysis (PCA) is an essential algorithm in machine learning. It is a mathematical method for evaluating the principal components of a dataset. The principal components are a set of vectors in high-dimensional space that capture the variance (i.e., spread) or variability of the feature space.

The goal of computing principal components is to find a low-dimensional feature sub-space that captures as much information as possible from the original higher-dimensional features of the dataset.

PCA is particularly useful for simplifying data visualization of high-dimensional features by reducing the dimensions of the dataset to a lower sub-space. For example, since we can easily visualize relationships on a 2-D plane using scatter diagrams, it will be useful to condense an n -dimensional space into two dimensions that retain as much information as possible in the n -dimensional dataset. This technique is popularly called dimensionality reduction.

How Are Principal Components Computed

The mathematical details for computing principal components are somewhat involved. This section will instead provide a conceptual but solid overview of this process.

The first step is to find the covariance matrix of the dataset. The covariance matrix captures the linear relationship between variables or features in the dataset. In a covariance matrix, an increasingly positive number represents a growing relationship, while the converse is represented by an increasingly negative number. Numbers around zero indicate a non-linear relationship between the variables. The covariance matrix is a square matrix (that means it has the same rows and columns). Hence, given a dataset with m rows and p columns, the covariance matrix will be a $m \times p$ matrix.

The next step is to find the eigenvectors of the covariance matrix dataset. In linear algebra theory, eigenvectors are non-zero vectors that merely stretch by a scalar factor, but do not change direction when acted upon by a linear transformation. We find the eigenvectors using a linear algebra technique called the singular value decomposition or SVD for short (see Figure 26-1). This advanced mathematical concept is beyond the scope of this book.

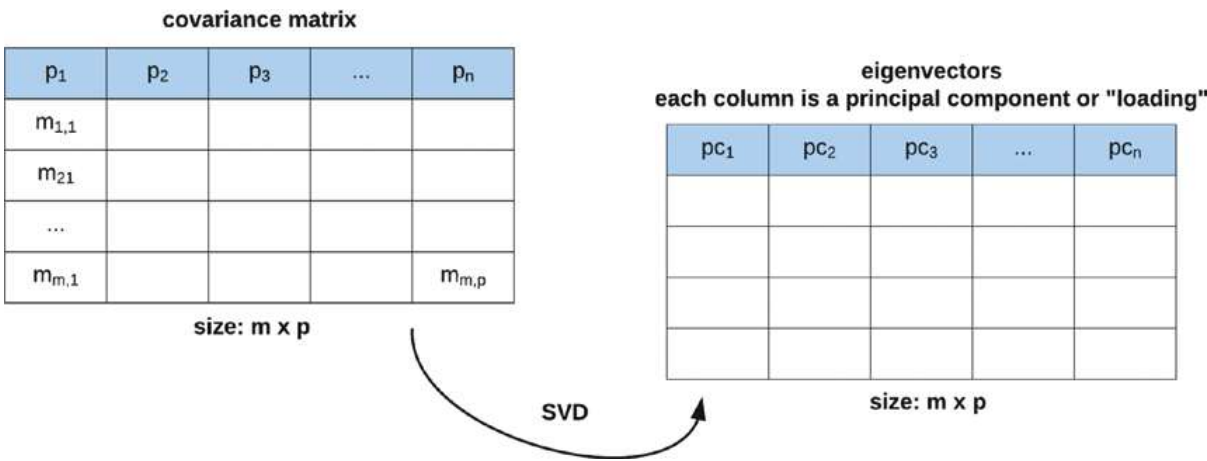


Figure 26-1. Decompose the covariance matrix using SVD to get the eigenvector matrix

The critical point to note at this junction is that the SVD also outputs a square matrix ($p \times p$), and each column of the matrix is an eigenvector of the original dataset. This output is the same across different software packages that compute the eigenvectors because the covariance matrix satisfies a mathematical property of being symmetric and positive semi-definite (the non-math inclined can conveniently ignore this point). We have as many eigenvectors as they are attributes or features in the dataset.

Without delving into mathematical theory, we can conclude that the eigenvectors are the principal components or loadings of the feature space. Again remember that the principal components capture the most significant variance in the dataset by projecting the data onto a vector called the first principal component. Other principal components are perpendicular to each other and capture the variance not explained by the first principal component. The principal components are arranged in order of importance in the eigenvector matrix, with the first principal component in the first column, the second principal component in the second column, and so on.

Dimensionality Reduction with PCA

To reduce the dimensions of the original dataset using PCA, we multiply the desired number of components or loadings from the eigenvector matrix, A , by the design matrix X . Suppose the design matrix (or the original dataset) has m rows (or observations) and p columns (or features), if we want to reduce the dimensions of the original dataset to two dimensions, we will multiply the original dataset X by the first two columns of the eigenvector matrix, $A_{reduced}$. The result will be a reduced matrix of m rows and 2 columns.

If X is a $m \times p$ matrix and $A_{reduced}$ is a $p \times 2$ matrix,

$$T_{reduced} = X_{m \times p} \times A_{p \times 2}$$

Observe that the result $T_{reduced}$ is a $m \times 2$ matrix. Hence, T is a 2-D representation of the original dataset X as shown in Figure 26-2.

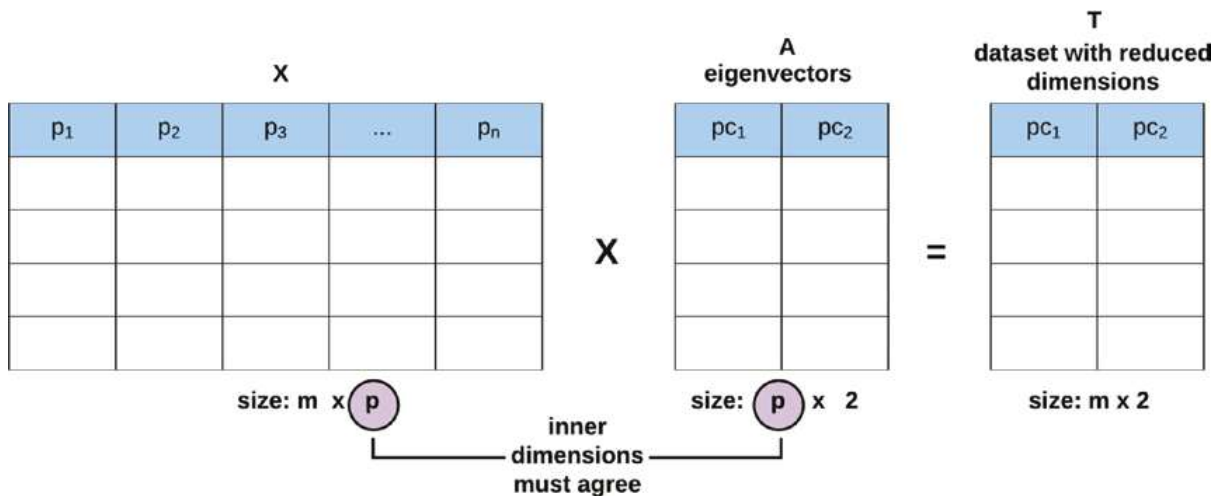


Figure 26-2. Reducing the dimension of the original dataset

In plotting the reduced dataset, the principal components are ranked in order of importance with the first principal component more prominent than the second and so on. Figure 26-3 illustrates a plot of the first two principal components.

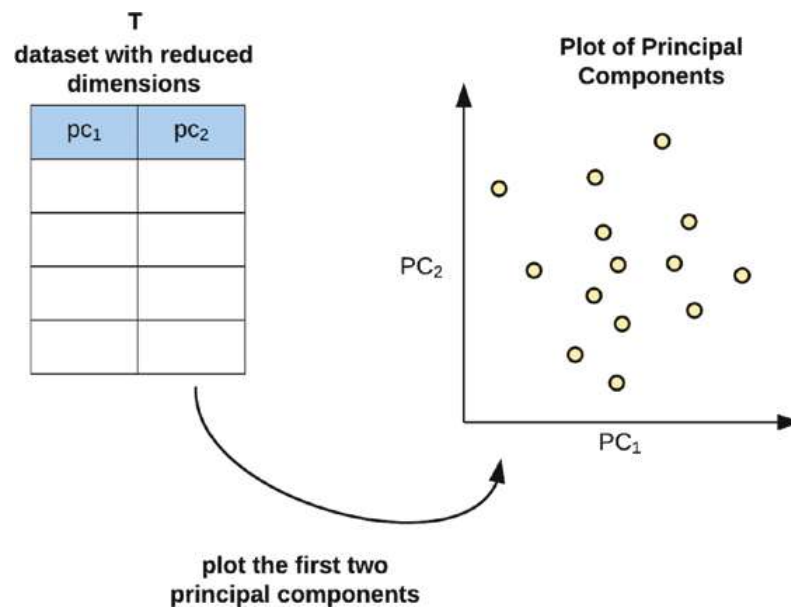


Figure 26-3. Visualize the principal components

Key Considerations for Performing PCA

It is vital to perform mean normalization and feature scaling on the variables of features of the original dataset before implementing PCA. This is because unscaled features can have stretched and narrow distance n-dimensional space, and this has a huge consequence when finding the principal components that explain the variance of the dataset (see Figure 26-4).

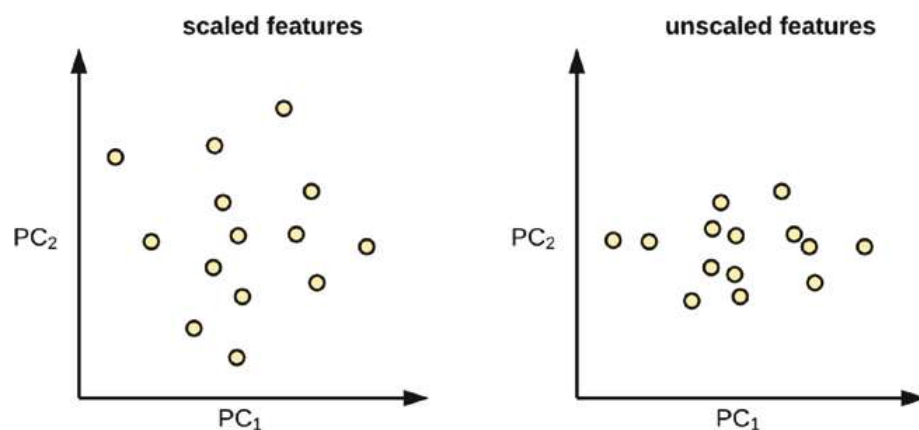


Figure 26-4. Right: An illustration of PCA with scaled features. Left: An illustration of PCA with unscaled features.

Again mean normalization ensures that every attribute or feature of the dataset has a zero mean, while feature scaling ensures all the features are within the same numeric range.

Finally, PCA is susceptible to vary wildly due to slight perturbations or changes in the dataset.

PCA with Scikit-learn

In this section, PCA is implemented using Scikit-learn.

```
# import packages
from sklearn.decomposition import PCA
from sklearn import datasets

from sklearn.preprocessing import Normalizer

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data

# normalize the dataset
scaler = Normalizer().fit(X)
normalize_X = scaler.transform(X)

# create the model.
pca = PCA(n_components=3)

# fit the model on the training set
pca.fit(normalize_X)

# examine the principal components percentage of variance explained
pca.explained_variance_ratio_

# print the principal components
pca_dataset = pca.components_
pca_dataset
'Output':
```

```
array([[ 0.18359702,  0.49546167, -0.76887947, -0.36004754],  
       [ 0.60210709, -0.64966313, -0.05931229, -0.46031175],  
       [-0.2436305 ,  0.28528504,  0.49319469, -0.78486663]])
```

In this chapter, we explained PCA giving a high-level overview of how it works to find a low-dimensional sub-space of a dataset. More so, we showed how PCA is implemented with Scikit-learn. This chapter concludes Part 4. In the next part, we introduce another scheme of learning methods called deep learning that builds on the machine learning neural network algorithm for learning complex representations.